

ValueSched

Perspective-Separated LLM Agents for Utility-Aware Dynamic GPU Scheduling

Research Plan for a Conference-Standard Trace-Driven Systems Study

Central thesis

A centralized LLM can generate diverse schedules, but it does not explicitly separate job-level marginal utility from cluster-level opportunity cost. Role-specialized demand and supply agents can expose these values before a formally constrained coordinator allocates resources.

Prepared from: The research discussion and the uploaded LLMSCHED paper
Document type: Conference-oriented research blueprint
Date: July 2026

Contents

1	Executive Summary	1
2	Motivation and Research Gap	1
3	Precise Problem Formulation	2
3.1	Job-Side Marginal Resource Utility	2
3.2	Cluster-Side Opportunity Cost	2
3.3	Allocation Objective	2
4	Proposed System Architecture	3
4.1	Demand-Side Valuation Agent	3
4.2	Supply-Side Valuation Agent	3
4.3	Negotiation Coordinator	4
4.4	Formal Verifier	4
5	Negotiation Protocol	4
6	Research Questions and Hypotheses	4
7	Trace-Driven Simulation Methodology	5
7.1	Pipelined Three-Minute Scheduling	5
7.2	Base Configuration	5
7.3	Counterfactual Performance Model	5
7.4	Reallocation Overhead	6
8	Baselines and Ablation Design	6
8.1	Essential Ablations	7
9	Evaluation Metrics	7
9.1	Standard Scheduling Outcomes	7
9.2	Metrics Specific to the Contribution	7
10	Experimental Regimes	8
11	Expected Conference Contributions	8
12	Threats to Validity and Mitigation	8

13 Implementation and Submission Roadmap	9
13.1 Minimum Viable Workshop Submission	9
13.2 Strong Regular Conference Submission	9
13.3 Top-Tier Strengthening Options	10
14 Draft Paper Framing	10
14.1 Working Title	10
14.2 Problem Statement Paragraph	10
14.3 Draft Abstract	10
15 Submission Readiness Checklist	10
16 Bottom-Line Assessment	11

1 Executive Summary

Research objective. Develop and rigorously evaluate an LLM-assisted scheduler that separates demand-side valuation from supply-side valuation before making dynamic GPU allocation decisions. The system targets malleable jobs in a trace-driven simulator with three-minute planning epochs.

Conference-ready claim

The contribution is not “three LLMs are better than one.” The contribution is a measurable valuation-and-allocation mechanism: job agents estimate marginal resource utility, a cluster agent estimates opportunity cost, a coordinator reconciles the two, and a formal solver guarantees feasibility.

Expected outcome. The study will test whether role-separated valuation lowers allocation regret, reduces over-allocation, improves useful GPU-hours, and preserves SLA and fairness objectives compared with equally funded centralized-LLM baselines.

Table 1: High-level study design.

Element	Planned design
System name	VALUESCHED
Decision model	Compute allocation A_{t+1} while executing A_t
Epoch duration	Three minutes; 200 ticks for a ten-hour episode
Primary dataset	Alibaba Cluster Trace V2020 windows
Initial workload scale	16 jobs per window, then 64 and 256 for scalability
Safety layer	Integer Linear Programming (ILP) or CP-SAT verifier
Primary scientific test	Equal-model, equal-token comparison with structured single-LLM scheduling

2 Motivation and Research Gap

Motivating baseline. The uploaded LLMSCHED paper uses a single scheduler-role LLM to generate multiple candidate allocations at different temperatures, then refines candidates with an ILP solver. This creates sampling diversity, but the candidates still arise from one centralized viewpoint. The paper also shows that constraint prompting reduces invalid assignments but does not eliminate them, making formal refinement essential [1].

$$\text{sampling diversity} \neq \text{perspective diversity.} \tag{1}$$

A centralized prompt asks one model to reason simultaneously about urgency, resource requests, scaling behavior, node capacity, fragmentation, locality, fairness, and SLA risk. In large states, context filtering and attention competition may cause the model to underweight information that determines the marginal value of an additional GPU.

Research gap

Existing LLM schedulers primarily generate placements or policies. They do not explicitly elicit two separate quantities before allocation: (1) the job-side benefit of additional resources and (2) the cluster-side opportunity cost of supplying them.

3 Precise Problem Formulation

Avoid the phrase “recover the true value of the data.” A trace records one realized execution and usually does not reveal every counterfactual runtime under every possible GPU allocation. The research should instead estimate a defined latent quantity and compare it with simulator-defined or profiled ground truth.

3.1 Job-Side Marginal Resource Utility

For an active job j at epoch t , define the utility of receiving g GPUs during the next epoch as

$$U_j(g, t) = \text{expected useful progress, JCT reduction, or SLA-risk reduction under allocation } g. \quad (2)$$

The main object of interest is the marginal benefit of one additional GPU:

$$\Delta U_j(g, t) = U_j(g + 1, t) - U_j(g, t). \quad (3)$$

Table 2: Illustrative job utility curve.

GPU allocation	Expected next-epoch progress	Marginal benefit
1 GPU	1.00 work units	—
2 GPUs	1.80 work units	0.80
3 GPUs	2.30 work units	0.50
4 GPUs	2.45 work units	0.15

3.2 Cluster-Side Opportunity Cost

For resource unit g on node n , define the opportunity cost of allocating it now as

$$C_n(g, t) = C_{\text{scarcity}} + C_{\text{fragmentation}} + C_{\text{locality}} + C_{\text{contention}} + C_{\text{reserve}}. \quad (4)$$

The scheduler should prefer an allocation only when its job-side marginal utility exceeds its system-side marginal cost after accounting for migration or resizing overhead.

3.3 Allocation Objective

The coordinator and optimizer target

$$\max_{A_{t+1}} \left[\sum_{j \in \mathcal{J}_t} \hat{U}_j(A_{j,t+1}, t) - \hat{C}(A_{t+1}, t) - \lambda H(A_t, A_{t+1}) \right], \quad (5)$$

where H measures allocation churn, checkpoint/restart cost, worker reconfiguration, and lost progress. The optimization remains subject to capacity, placement, minimum allocation, fairness, and malleability constraints.

4 Proposed System Architecture

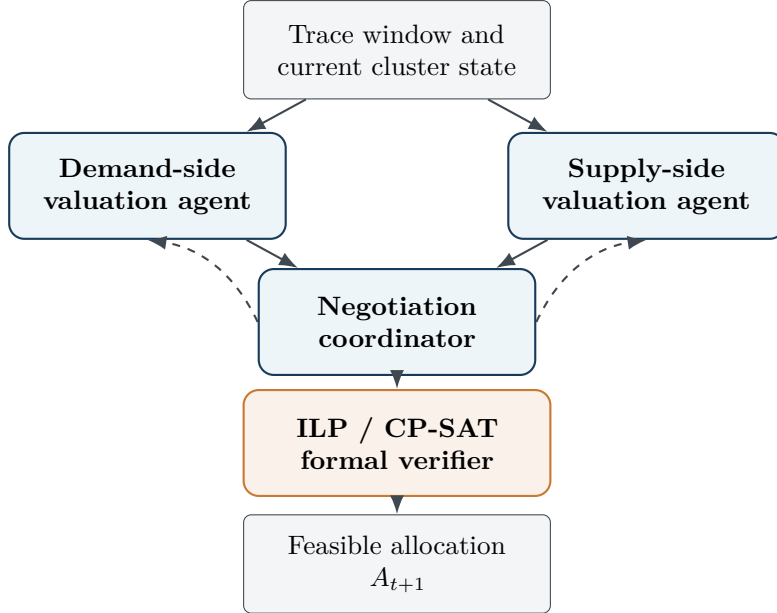


Figure 1: VALUESCHED separates demand and supply valuation, permits structured revision, and uses a formal verifier before activation.

4.1 Demand-Side Valuation Agent

The demand agent represents active jobs. It must output a machine-readable valuation record rather than a free-form request.

Table 3: Demand-agent output schema.

Field	Meaning
$g_j^{\min}$	Minimum feasible allocation
$g_j^{\max}$	Maximum useful allocation
$\hat{U}_j(g, t)$	Estimated utility curve over feasible GPU counts
Slack / urgency	Deadline risk or slowdown priority
Resize cost	Checkpoint, restart, reconfiguration, or migration penalty
Confidence	Calibrated certainty with cited input evidence

4.2 Supply-Side Valuation Agent

The supply agent represents the cluster. It evaluates free and occupied resources, GPU-memory coupling, placement compatibility, fragmentation, data locality, expected contention, and the value

of retaining spare capacity. Its output is a structured marginal-cost or price curve, not merely a list of free nodes.

4.3 Negotiation Coordinator

The coordinator receives demand bids and supply costs, constructs a provisional allocation, exposes conflicts, and requests at most one revision round. The same underlying model should be used for all roles in the primary study. A stronger coordinator model can be evaluated later as a separate factor.

4.4 Formal Verifier

An ILP or CP-SAT layer converts the negotiated proposal into an executable allocation. It guarantees capacity, compatibility, minimum allocation, maximum useful allocation, fairness, affinity, anti-affinity, GPU-memory coupling, and reallocation limits. This layer creates a clean separation between semantic reasoning and hard correctness.

5 Negotiation Protocol

1. **Round 1 – demand bids:** each job submits a marginal-utility curve, urgency, resize cost, and confidence.
2. **Round 1 – supply prices:** the cluster submits scarcity, fragmentation, locality, and opportunity-cost estimates.
3. **Provisional allocation:** the coordinator creates $A^{(1)}$ and explains the highest-value trade-offs.
4. **Revision round:** each side receives the provisional result and may revise low-confidence or conflicting estimates once.
5. **Final proposal:** the coordinator creates $A^{(2)}$ and sends it to the formal verifier.
6. **Activation:** the feasible allocation becomes A_{t+1} at the next tick boundary.

Terminology rule

If the method uses only one message from each side, call it *role decomposition* or *perspective separation*. Use the term *negotiation* only when the agents receive a provisional allocation and can revise their positions.

6 Research Questions and Hypotheses

Research question	Hypothesis
RQ1. Does perspective separation improve valuation accuracy?	H1. It lowers marginal-utility MAE and improves ranking of the next best GPU recipient.

Research question	Hypothesis
RQ2. Does improved valuation improve allocation quality?	H2. It lowers allocation regret, over-allocation, and wasted GPU-hours.
RQ3. Are gains caused by role separation rather than extra computation?	H3. Gains remain under equal model, token, call, and solver budgets.
RQ4. When does negotiation help most?	H4. Benefits are largest under contention, inaccurate requests, heterogeneous scaling, and tight deadlines.
RQ5. Is the method operationally practical?	H5. End-to-end planning finishes within 180 seconds and its benefit exceeds inference cost.

7 Trace-Driven Simulation Methodology

7.1 Pipelined Three-Minute Scheduling

During epoch t , jobs execute under allocation A_t while the scheduler prepares A_{t+1} :

$$\underbrace{\text{execute } A_t}_{[t, t+180 \text{ s})} \parallel \underbrace{\text{estimate values, negotiate, and verify } A_{t+1}}_{T_{\text{decision}} \leq 180 \text{ s}}. \quad (6)$$

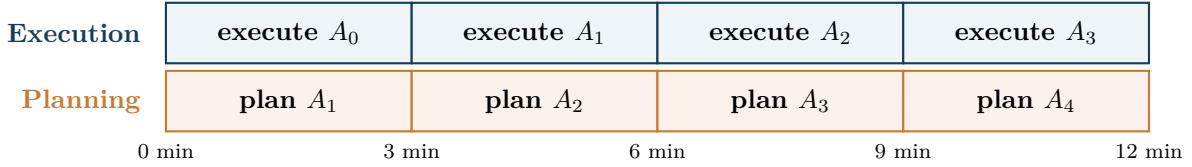


Figure 2: At epoch t , execution under A_t proceeds in parallel with planning for A_{t+1} .

7.2 Base Configuration

7.3 Counterfactual Performance Model

Alibaba V2020 supplies arrivals, requests, observed utilization, and realized durations, but it does not directly provide the runtime of every job under every alternative GPU count. A conference-quality study therefore needs a calibrated simulator-side scaling model. One candidate is

$$P_j(g) = P_j(1) \frac{g}{1 + \alpha_j(g - 1)}, \quad (7)$$

where α_j captures serial and communication overhead. It should be estimated by job class where possible, validated against representative GPU profiling, and varied in sensitivity analysis. The paper must clearly label trace-observed fields, estimated fields, and simulator-defined ground truth.

Table 5: Base simulator configuration.

Parameter	Planned value
Episode duration	10 hours
Ticks per episode	200
Tick length	180 seconds
Base jobs per trace window	16
Scalability workloads	64 and 256 jobs
Scheduler invocation	Every tick
Global reallocation	Tick boundaries only
Future-arrival knowledge	Not allowed unless a predictive or oracle baseline is explicitly labeled
Independent windows / seeds	At least 30; preferably 100
Statistics	Mean, median, 95% confidence interval, paired tests, and effect sizes

7.4 Reallocation Overhead

Resource resizing is not free. The simulator should account for checkpoint time, process restart, worker reconfiguration, data reload, and lost progress. The main evaluation should compare a zero-cost upper bound with one or more realistic overhead models.

8 Baselines and Ablation Design

Method	Purpose
Requested-resource allocation	Measures the cost of trusting user requests.
Traditional heuristic (e.g., DRF, Tetris, best-fit, priority scheduling)	Tests whether LLM reasoning is needed.
LLMSCHED-style single scheduler role	Provides a direct single-LLM comparison motivated by the uploaded paper.
Structured single LLM	Receives the same demand/supply schema; tests whether role separation is necessary.
Same-role LLM ensemble	Controls for candidate diversity without demand-supply specialization.
Demand + supply, no revision	Measures one-shot perspective decomposition.
Demand + supply + coordinator negotiation	Measures the value of revision and reconciliation.
Full VALUESCHED + formal solver	Evaluates the complete safe system.
Oracle optimizer	Uses simulator ground-truth utilities to define regret.

Critical fairness control

The structured single-LLM baseline must receive the same information and an equivalent inference budget. For example, compare three 4,000-token role calls with either one 12,000-token centralized call or three centralized calls using the same model version and solver.

8.1 Essential Ablations

- Remove the supply agent.
- Remove the demand agent.
- Remove the revision round.
- Replace the coordinator with a deterministic merger.
- Remove confidence estimates.
- Remove the churn penalty.
- Remove the formal verifier.
- Use a stronger coordinator model only as a separate secondary experiment.

9 Evaluation Metrics

9.1 Standard Scheduling Outcomes

- Average, median, and P95 job completion time and slowdown.
- GPU, CPU, and memory utilization.
- SLA violation rate and deadline-miss severity.
- Jain fairness index or dominant-share fairness.
- Allocation churn, migration frequency, and resizing overhead.

9.2 Metrics Specific to the Contribution

Metric	Definition / interpretation
Marginal-utility MAE	Average $ \hat{U}_j(g) - U_j^*(g) $ over jobs and feasible GPU counts.
Ranking accuracy	Frequency with which the scheduler identifies the job with the highest true next-GPU benefit.
Allocation regret	$V(A^*) - V(A_{\text{scheduler}})$ using simulator ground truth.
Useful GPU-hour ratio	GPU time that produces useful modeled progress divided by total allocated GPU time.
Over-allocation	Resources assigned beyond the job-specific useful allocation.
Under-allocation	Resources denied to jobs with high marginal benefit.

Metric	Definition / interpretation
Proposal feasibility	Fraction of LLM proposals accepted, repaired, or rejected by the formal solver.
Negotiation overhead	Wall time, tokens, messages, monetary cost, and fraction of decisions exceeding 180 seconds.

10 Experimental Regimes

Regime	What it tests
Underloaded	Whether the method avoids unnecessary complexity when resources are abundant.
Normal load	Steady-state scheduling quality.
High contention	Ability to identify the highest marginal use of scarce GPUs.
Bursty arrivals	Robustness to abrupt state changes and stale next-tick plans.
Inaccurate user requests	Ability to distinguish requested resources from useful resources.
Heterogeneous scaling	Need for job-specific utility curves.
Tight deadlines	Priority and SLA trade-offs.
High resizing cost	Whether the scheduler avoids unstable allocations.
Distribution shift	Generalization across unseen job mixtures and scaling parameters.

11 Expected Conference Contributions

1. A formalization of dynamic GPU scheduling as perspective-separated valuation: job-side marginal utility versus cluster-side opportunity cost.
2. A structured demand–supply negotiation protocol with confidence, evidence, and one revision round.
3. A hybrid LLM-plus-optimization architecture that preserves semantic reasoning while guaranteeing feasibility.
4. A trace-driven evaluation framework with calibrated counterfactual scaling and explicit reallocation overhead.
5. An equal-budget study that distinguishes role specialization from extra tokens, extra calls, and stronger models.
6. New metrics for LLM scheduling: valuation error, allocation regret, useful GPU-hours, and negotiation overhead.

12 Threats to Validity and Mitigation

Threat	Mitigation
Counterfactual utility is not directly observed in the trace	Calibrate by job class, profile representative workloads, expose assumptions, and run sensitivity analysis.
Multi-agent gains may come from more inference compute	Use equal-model, equal-token, and equal-call baselines.
Personas may only reformat the same information	Compare against a structured single LLM receiving the same schema.
LLM estimates may be unsupported	Require evidence fields, confidence scores, and machine-checkable numeric ranges.
Agent communication may exceed the epoch deadline	Run demand and supply agents in parallel, cap one revision round, and use timeouts and fallbacks.
Sixteen jobs may be too small for broad conclusions	Use many windows and include 64- and 256-job scaling experiments.
Trace simulation may not reflect real resizing behavior	Validate a subset on actual GPU workloads or published scaling profiles.

13 Implementation and Submission Roadmap

Phase	Main outputs	Duration
1. Simulator foundation	Alibaba window extractor, event model, baseline scheduler, three-minute pipeline	3–4 weeks
2. Ground-truth utility layer	Job-class scaling curves, reallocation-cost models, oracle optimizer	3–5 weeks
3. VALUESCHED prototype	Structured schemas, parallel agents, coordinator, formal verifier	4–6 weeks
4. Controlled ablations	Equal-budget single-LLM and ensemble baselines, prompt-stability tests	3–4 weeks
5. Full evaluation	30–100 windows, workload regimes, statistical tests, scalability	4–6 weeks
6. Validation and writing	Real-workload spot validation, figures, limitations, paper draft	4–6 weeks

13.1 Minimum Viable Workshop Submission

Sixteen-job windows, at least 30 seeds, a defensible utility model, an equal-budget single-LLM baseline, VALUESCHED with formal verification, and regret/utilization/JCT results.

13.2 Strong Regular Conference Submission

Multiple workload scales, calibrated counterfactual models, realistic reallocation overhead, broad ablations, statistical significance, cost/latency analysis, and real-workload validation.

13.3 Top-Tier Strengthening Options

- A theoretical bound or convergence property for the negotiation mechanism.
- A real cluster prototype with malleable training or inference jobs.
- Evaluation across multiple traces or workload families.
- A learned or calibrated valuation model that reduces dependence on prompt-only estimates.

14 Draft Paper Framing

14.1 Working Title

ValueSched: Perspective-Separated LLM Agents for Utility-Aware Dynamic GPU Scheduling

14.2 Problem Statement Paragraph

Existing LLM-assisted schedulers encode heterogeneous workload and cluster information into a unified prompt and use a centralized language model to generate candidate allocations. Although repeated sampling can produce diverse candidates, it does not explicitly distinguish between a job’s marginal benefit from additional resources and the cluster’s opportunity cost of supplying those resources. This limitation is important for malleable GPU workloads, where user requests can overstate useful demand and where the value of an additional GPU varies across jobs and epochs. We therefore study whether perspective-separated demand and supply agents can elicit more accurate allocation utilities before a formally constrained scheduler makes the final decision.

14.3 Draft Abstract

Dynamic allocation of GPU resources requires a scheduler to determine not only whether resources are available, but also where each resource produces the greatest marginal benefit. Existing LLM-assisted schedulers use centralized language-model reasoning to generate candidate placements, combining job demands, cluster conditions, and scheduling objectives in a single context. We present VALUESCHED, a perspective-separated framework in which demand-side agents estimate job-level marginal utility, supply-side agents estimate cluster-level scarcity and fragmentation costs, and a coordinator reconciles both views through a structured negotiation protocol. A formal optimization module converts the negotiated proposal into a feasible allocation while accounting for capacity, placement, malleability, fairness, and reallocation overhead. We evaluate VALUESCHED in a trace-driven simulator based on Alibaba V2020 workloads using three-minute planning epochs. Beyond job completion time, utilization, and SLA violations, we measure marginal-utility estimation error, useful GPU-hours, allocation regret, and negotiation overhead. Equal-budget comparisons with centralized single-LLM, LLM-ensemble, heuristic, and oracle baselines isolate the value of perspective separation from additional inference computation.

15 Submission Readiness Checklist

- ☐ The utility definition is explicit and simulator ground truth is reproducible.

- ☐ The structured single-LLM baseline receives the same information and inference budget.
- ☐ Future trace arrivals are hidden from non-oracle schedulers.
- ☐ Reallocation overhead is modeled and sensitivity-tested.
- ☐ The formal solver guarantees every executed allocation.
- ☐ At least 30 independent windows are evaluated; preferably 100.
- ☐ Results include confidence intervals, effect sizes, and paired statistical tests.
- ☐ The paper reports both scheduling quality and inference cost.
- ☐ The study identifies conditions under which VALUESCHED does not help.
- ☐ The conclusion claims perspective-separated valuation, not generic multi-agent superiority.

16 Bottom-Line Assessment

Research potential

The direction is strong enough for a serious conference project when its novelty is framed as explicit utility elicitation and opportunity-cost reasoning, not simply the use of multiple LLMs. The decisive experiment is whether perspective separation reduces valuation error and allocation regret against an equally funded, well-structured single LLM.

A workshop-quality version is achievable with a rigorous simulator and controlled baselines. A strong regular conference paper requires credible counterfactual utility, realistic resizing cost, many trace windows, equal-compute ablations, and formal feasibility. A top-tier systems submission likely also needs real-cluster validation, stronger mechanism design, or theoretical insight.

References

- [1] G. Ding, S. Yang, H. Lin, Z. Chen, and J. S. Yang, “LLM-Driven Adaptive Cloud Resource Scheduling: Bridging Reasoning Intelligence With Optimization Guarantees,” *IEEE Open Journal of the Computer Society*, vol. 7, 2026. DOI: [10.1109/OJCS.2026.3667549](https://doi.org/10.1109/OJCS.2026.3667549).
- [2] Alibaba Cluster Trace Program, “Alibaba Cluster Trace V2020 Dataset.” Dataset fields and preprocessing assumptions should be documented precisely in the final paper.